Key-Value Store (with transactions)

Problem Statement

Implement SET/GET/DELETE/COUNT with optional BEGIN/ROLLBACK/COMMIT transactions.

Variants

- Versioned GET; TTL

- Nes

How to Leverage the Solution

Use maps for values and value-counts; transaction stack holds undo ops for rollback/commit.

Java Solution

Below is a concise reference implementation. Full comments omitted for brevity.

```
import java.util.*;

public final class KeyValueStore {
 private final Map<String, String> store = new HashMap<>();
 private final Map<String, Integer> valueCounts = new HashMap<>();
 private final Deque<List<Runnable>> txStack = new ArrayDeque<>();
 public void set(String key, String value) {
  String old = store.put(key, value); if (old != null) decrement(old); increment(value);
  recordUndo(() -> { String prev = old; String cur = store.put(key, prev); if (cur != null) decreme
nt(cur); if (prev != null) increment(prev); });
 }
 public String get(String key) { return store.get(key); }
 public void delete(String key) { String old = store.remove(key); if (old == null) return; decremen
t(old); recordUndo(() -> { String cur = store.put(key, old); if (cur != null) decrement(cur); incre
ment(old); }); }
 public int count(String value) { return valueCounts.getOrDefault(value, 0); }
 public void begin() { txStack.push(new ArrayList<>()); }
 public boolean rollback() { if (txStack.isEmpty()) return false; List<Runnable> undos = txStack.po
p(); for (int i = undos.size() - 1; i >= 0; i--) undos.get(i).run(); return true; }
 public boolean commit() { if (txStack.isEmpty()) return false; List<Runnable> changes = txStack.po
p(); if (!txStack.isEmpty()) txStack.peek().addAll(changes); return true; }
 private void increment(String value) { valueCounts.put(value, valueCounts.getOrDefault(value, 0) +
 1); }
 private void decrement(String value) { valueCounts.compute(value, (k, v) -> v == null or v <= 1 ?
null : v - 1); }
 private void recordUndo(Runnable undo) { if (!txStack.isEmpty()) txStack.peek().add(undo); }
}
```